

ARGOS

Adaptive Response Guard with Orchestrated Surveillance

Manual del integrante

Diego Jara

Infraestructura · UI · Evaluación

Entrega final: 13 de junio de 2026

Universidad de Lima · Tópicos Avanzados de Ciberseguridad · 2026-1

Versión 3.0 — manual organizado por orden de implementación

Parte I — Introducción común al proyecto

ARGOS — Introducción al proyecto y arquitectura

Documento común a todos los manuales de integrante. Contiene el contexto que cada miembro del equipo (P1/P2/P3/P4) necesita antes de leer su manual individual.

Campo	Valor
Tipo	Introducción común para los 4 integrantes
Estado	Activo
Entrega final	13 de junio de 2026 (sábado)
Pre-requisito	Ninguno. Léelo de cero.

1. Qué es ARGOS en una página

ARGOS es la **Adaptive Response Guard with Orchestrated Surveillance** — una plataforma multi-vector de detección y respuesta a amenazas que replica la arquitectura de productos comerciales high-end EDR/XDR (Microsoft Defender XDR, CrowdStrike Falcon, Palo Alto Cortex XDR) usando exclusivamente componentes open source más una API LLM de bajo costo para la capa de triage.

El proyecto es parte del curso **Tópicos Avanzados de Ciberseguridad** en la Universidad de Lima · 2026-1. La entrega final es el **13 de junio de 2026** y consta de tres deliverables obligatorios: informe técnico (~30 % del peso), demo en vivo (~40 %), y presentación (~20 %). Los seguimientos intermedios pesan el ~10 % restante.

El **activo defendido** es una base de datos **PostgreSQL 15** corriendo sobre una Linux VM en el lab. Tiene esquema `argos_demo_prod` con tablas que simulan datos de RRHH y finanzas (employees, payroll, customers, invoices, payments) con datos sintéticos. El host está tagged en Wazuh como

`criticality=production-critical`, lo que dispara la regla de dos personas (two-person rule) para cualquier acción de containment sobre él.

El **alcance multi-vector** (post ADR-0008) cubre tres categorías de ataques distintas:

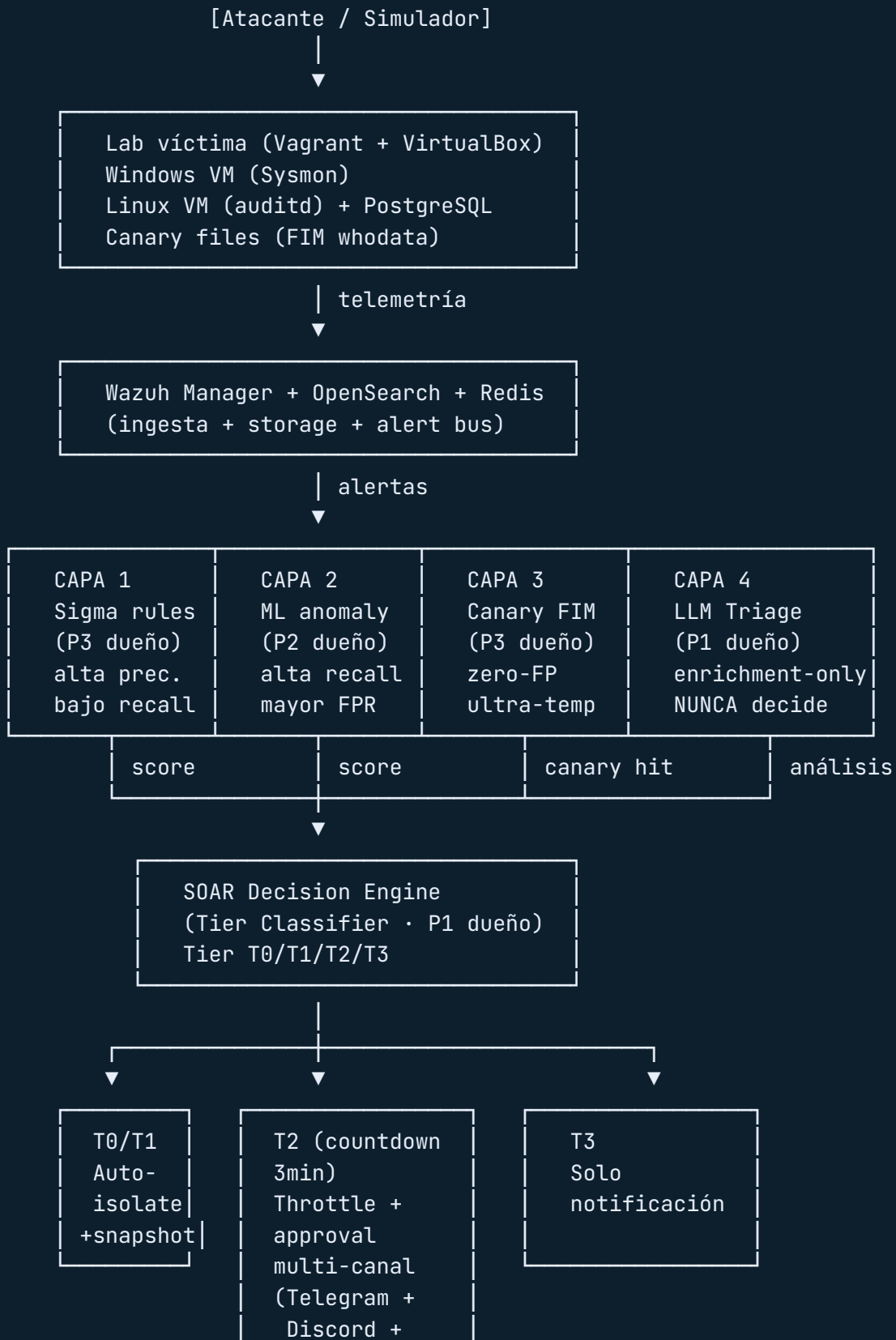
- **Ransomware:** cifrado de archivos via técnicas T1486/T1490/T1083/T1562. Es el énfasis primario.
- **Network Denial of Service:** ataques de red contra el puerto del servicio (T1498/T1499) con `hping3` o `slowhttptest`.
- **Application Abuse:** SQL injection contra una app web delante de la DB (T1190) usando `sqlmap`, y false positives de actividad legítima de usuario (T1078 Valid Accounts) — un SELECT masivo a las 3 AM se parece a exfiltración pero puede ser un reporte mensual legítimo.

Por qué este alcance y no más: ARGOS no busca cobertura exhaustiva multi-vector. Busca cobertura mínima representativa que demuestre que la arquitectura de 4 capas + SOAR + HITL es genuinamente adaptativa. Cubrir 3 vectores con profundidad real es más defendible que cubrir 10 superficialmente.

2. Arquitectura de 4 capas defensivas

ARGOS sigue el patrón industrial estándar de **defense-in-depth**: cuatro capas independientes y paralelas, cada una con trade-offs distintos, fallan independientemente. Si una capa cae o es evadida, las otras tres mantienen cobertura. No hay un solo punto de falla en la detección.





Capa 1 — Rule-Based Detection (Sigma + Wazuh)

Dueño: P3 (Angeles). Reglas YAML escritas en formato Sigma, convertidas a Wazuh con `sigma-cli`, mapeadas explícitamente a técnicas MITRE ATT&CK. Detectan patrones conocidos.

Sub-categorías por dominio (post ADR-0008):

- `detection/sigma-rules/ransomware/` — vssadmin, T1486 cifrado, T1083 enumeración, T1562 disable defender, T1021 lateral, T1071 C2
- `detection/sigma-rules/network/` — rate-based rules para T1498/T1499 (DDoS, slow-rate)
- `detection/sigma-rules/database/` — query pattern anomalies para UC-07
- `detection/sigma-rules/webapp/` — T1190 SQL injection signatures

Trade-off honesto: alta precisión, recall limitado contra variantes nuevas. Por eso existe Capa 2.

Capa 2 — ML Anomaly Detection

Dueño: P2 (Sebastian). Modelos no supervisados entrenados sobre baseline benigno. Detectan desviaciones que las reglas no captan.

Modelos especializados por dominio (post ADR-0008):

- `ml/models/ransomware_ensemble.pkl` — Isolation Forest + One-Class SVM. Features ventana 60s: `file_write_rate`, `avg_entropy` (Shannon), `extension_modification_ratio`, `crypto_api_calls`, `new_outbound_connections`, `cpu_burst_score`, `io_burst_score`.
- `ml/models/network_traffic_anomaly.pkl` — para UC-06 DDoS. Features: `connections/sec`, `packet rate`, `source IP entropy`, `packet size variance`.
- `ml/models/query_pattern_anomaly.pkl` — para UC-07 SELECT masivo. Features: `rows_returned`, `query_duration_ms`, `hour_of_day`, `user_id`, `query_template_hash`.

Ensemble ransomware: $\text{ensemble_score} = 0.6 \times \text{isolation_forest_score} + 0.4 \times \text{one_class_svm_score}$. Los demás modelos retornan score único.

Trade-off honesto: mayor recall contra variantes nuevas, mayor false positive rate. Requiere baseline limpio.

Capa 3 — Canary Deception

Dueño: P3 (Angeles). Archivos-cebo (`financials_Q4_2025.xlsx` , `passwords.txt` , `db_backup.sql`) en ubicaciones donde un usuario legítimo nunca tocaría. FIM (File Integrity Monitoring) con Sysmon whodata (Windows) o auditd (Linux) captura QUÉ proceso accedió.

Lógica: primera modificación / lectura / rename de un canary = alerta crítica con confianza máxima. Por diseño, FP rate ≈ 0 .

Esta capa es estrictamente ransomware-specific. No tiene sub-categorías por dominio. Documentado deliberadamente en ADR-0008: defender contra DDoS o SQL injection requiere otras primitivas, no canaries.

Capa 4 — LLM-Assisted Triage

Dueño: P1 (Enzo). FastAPI service que recibe el contexto completo de una alerta (process tree, network connections, file modifications) y produce análisis estructurado: técnica MITRE, severidad, runbook NIST aplicable, acción recomendada, IoCs a correlar.

Backend vendor-agnostic (per ADR-0001 v2):

- **Primary:** OpenAI GPT-4o-mini (US-based, soberanía de datos aceptable)
- **Fallback:** Llama 3.1 8B local vía Ollama (zero-egress, funciona sin internet)

Invariante crítico R-02: el LLM **NUNCA** está en el **path crítico de containment**. El SOAR decide desde Capas 1-3 solamente. El LLM enriquece la vista del analista; si alucina, miente, o cae, el sistema sigue funcionando.

Pipeline RAG: BM25 (léxico) + BGE-large embeddings (denso) + RRF (Reciprocal Rank Fusion). Hybrid retrieval estándar industrial. Sin cross-encoder reranker (descartado en ADR-0001 v2 por marginal gain vs costo).

3. SOAR Decision Engine y los 4 tiers de confianza

El **SOAR (Security Orchestration, Automation and Response)** es el cerebro que fusiona señales de las 4 capas y decide la acción. Lo dueño es **P1**. Implementa la lógica de tiers per ADR-0003.

Tier	Disparado por	Confianza	Acción
 T0	Canary solo, OR las 3 capas corroboran	≥ 0.95	Aislamiento automático inmediato + snapshot
 T1	Layer 1 + Layer 2 corroboran (sin canary)	0.80–0.95	Aislamiento automático inmediato + snapshot
 T2	Capa sola con score alto	0.60–0.80	Throttle + snapshot ahora , aislamiento full pendiente de aprobación 3-min
 T3	Corroboración baja	0.40–0.60	Solo notificación enriquecida con LLM

(*) **Los thresholds son preliminares** pendientes de calibración empírica per `OPEN_QUESTIONS_RESOLUTION.md` §Q5.

Tier T2 — el más interesante

Ransomware moderno cifra ~25,000 archivos/min. Un countdown de 3 minutos para aprobación humana sin protección parece estúpido. Por eso T2 NO es "pendiente": es "throttle + snapshot AHORA, full isolation pendiente de aprobación". El throttle (cpulimit/ionice) reduce la tasa de cifrado a ~100-500 files/min mientras los aprobadores ven el contexto LLM y deciden. Si el timeout de 3 min expira sin respuesta, el sistema auto-ejecuta (no espera más). Si el aprobador clickea Reject, el throttle se levanta y el caso queda registrado como false positive con razón documentada — esto es UC-07.

Split-brain (conflicto multi-aprobador)

Per ADR-0006: cuando hay 4 aprobadores y dan respuestas contradictorias (2 approve, 1 reject, 1 timeout), se aplica **conservative-wins policy** con ventana de consolidación de 60 segundos: cualquier "approve" gana sobre rechazos, excepto para acciones irreversibles que requieren **two-person rule** (dos approves explícitos, un reject cancela).

El **two-person rule** se activa cuando el host afectado tiene `criticality=production-critical` (nuestro PostgreSQL). Esto es UC-04 en el demo.

Canales de notificación (per ADR-0007 v2)

Tiempo	Canal	Propósito
t=0	Telegram bot con botones inline JWT	Primario — push agresivo a celulares de aprobadores
t=0	Discord webhook con @-mention de role	Visibilidad en server del equipo, presión social
t=60s	Twilio Voice DTMF (sin STT)	Escalación si nadie respondió. "Presione 1 para aprobar, 2 para rechazar"
post-decisión	Email	Resumen asíncrono, NUNCA en path crítico

4. Equipo y responsabilidades

	Integrante	Rol	Alcance principal
	Enzo Ordoñez Flores	P1 · Líder · LLM/ SOAR	<code>argos_contracts</code> (entregado), Capa 4 LLM Triage, motor SOAR + Tier Classifier, Approval API con JWT, notificaciones multi-canal, Consola de Aprobación Streamlit, simulador de ransomware, playbooks de containment, coordinación
	Sebastian Montenegro	P2 · Ingeniero ML	Capa 2 completa (Isolation Forest + One-Class SVM ensemble + modelos especializados), feature extraction, calibración thresholds, métricas P/R/F1, captura forense
	Angeles Castillo	P3 · Detección y Engaño	Capa 1 (Sigma rules ransomware + network + database + webapp), Capa 3 (canary FIM + whodata), validación con Atomic Red Team y Caldera, bonus: PRs upstream a SigmaHQ
	Diego Jara	P4 · Infraestructura	Vagrantfile + Wazuh/OpenSearch/Redis deployment, PostgreSQL con datos sintéticos, simuladores (ransomware + DDoS + SQL injection), UI Streamlit base, video demo

Regla operativa crítica: cada integrante debe poder defender su capa en exposición. P1 no escribe código de otros con Claude Code. Cada integrante puede usar Claude Code (o cualquier asistente IA) como **acelerador en SU propia parte**, siempre que entienda lo que produce y pueda defenderlo en viva (per CONTEXT.md §5 reinterpretado post ADR-0008).

5. Los 8 use cases del demo

El demo en vivo cubre 8 escenarios (per ADR-0008 multi-vector). Cada UC tiene narrativa específica que demuestra una propiedad distinta del sistema.

UC	Vector	Tier	Capas que firing	Qué demuestra
UC-01	Ransomware (LockBit-like)	T0	1+2+3	Full-stack end-to-end. Las 3 capas firing simultáneo. Auto-isolate + email post-facto.
UC-02	Canary deception	T0	3 sola	Detección ultra-temprana. Zero archivos reales cifrados.
 UC-03	Variante novel + split-brain	T2	2 sola	Centerpiece HITL ransomware. ML atrapa lo que reglas no. 4 aprobadores. Conservative-wins live.
UC-04	PostgreSQL + two-person rule	T1	1+2	Compliance vocabulary. Four-eyes principle. Governance.
UC-05	Stealth attack (agent-kill)	T0	1+3+heartbeat	Resiliencia: agent disconnect = signal.
UC-06	DDoS volumetric	T0	1 (rate)+2 (network)	Cobertura multi-vector. Defiende contra ataques de red, no solo endpoint. T1498.
 UC-07	SELECT masivo legítimo FP	T2	2 sola	Pieza clave del HITL. Humano cancela contención por FP reconocido. T1078.
UC-08	SQL injection	T1	1+2	OWASP Top 10 #1. T1190 Exploit Public-Facing Application.

Los dos  son los UCs que más venden el HITL. UC-03 muestra split-brain con conservative-wins; UC-07 muestra cancelación de FP por humano. Ambos son lo que diferencia un EDR/XDR con HITL de un SIEM tradicional.

6. Ejemplo end-to-end de UC-01 paso a paso

Para que tengas un modelo mental concreto de cómo fluye una alerta a través del sistema, aquí está UC-01 (ransomware clásico LockBit-like) desglosado segundo a segundo.

Setup pre-ataque (T-5 min)

- Lab está corriendo: Wazuh manager + Windows VM + Linux VM con PostgreSQL.
- Los 4 aprobadores tienen Telegram abierto en sus celulares.
- La pantalla del proyector muestra: Streamlit Approval Console (vacía) + terminal de P4 listo para ejecutar el simulador.
- P1 narra el contexto: "Este es nuestro endpoint Windows típico de la organización. Tiene documentos en `C:\Users\Demo\Documents\`. El atacante acaba de obtener acceso inicial."

T=0 — Atacante ejecuta

P4 corre:



BASH

```
python attack-simulation/ransomware_simulator/lockbit_like.py --target windows-victim
```

Esto inicia el simulador. Behavior chain:

1. T+1s: enumera archivos en `C:\Users\Demo\Documents\` (técnica T1083 File and Directory Discovery)
2. T+2s: invoca `vssadmin delete shadows /all /quiet` (técnica T1490 Inhibit System Recovery)
3. T+3s: comienza a cifrar archivos con AES-256, renombrando a `.locked` (técnica T1486 Data Encrypted for Impact)
4. T+4s: toca el canary file `financials_Q4_2025.xlsx`
5. T+5s: deja la ransom note `README_RESTORE_FILES.txt` en el desktop

T+1 a T+5 segundos — Capas detectan en paralelo

Capa 1 (Sigma + Wazuh) detecta:

- T+2s: regla `T1490_vssadmin_delete_shadows` dispara cuando ve el comando `vssadmin`.
- T+3s: regla `T1486_mass_file_encryption_extension` dispara al ver renames masivos a `.locked`.

- Wazuh manager normaliza estas alertas y las publica al stream Redis `wazuh:alerts` .

Capa 2 (ML) detecta:

- ML consumer (Python proceso corriendo en bg) lee el stream Redis.
- Cada 60s genera features para los procesos activos. La ventana T+1 a T+60 captura el simulator process.
- Features observadas: file_write_rate=347 archivos/min, avg_entropy=7.8 (alta), extension_modification_ratio=0.95, crypto_api_calls=42, cpu_burst_score=0.9.
- Isolation Forest score: 0.94. One-Class SVM score: 0.91. Ensemble: 0.93.
- Publica `MLScore` con score 0.93 al stream Redis `ml:scores` .

Capa 3 (Canary FIM) detecta:

- T+4s: el simulator toca `financials_Q4_2025.xlsx` .
- Sysmon whodata captura el evento con PID del simulator y command-line completo.
- Wazuh dispara regla custom `canary_file_accessed` con severity 12 (crítica).
- Score implícito: 1.0 (canary fire = certeza máxima).

T+5 segundos — SOAR Decision Engine fusiona

El **SOAR orchestrator** (proceso de P1 corriendo el FastAPI Approval API en port 8003) se entera de las 3 alertas dentro del mismo incident window de 5 segundos.

Tier Classifier (`soar/decision_engine/tier_classifier.py`) aplica reglas:

- `canary_fired=True` → tier T0 (canary alone wins to T0).
- Pero también L1 + L2 corroboran con high scores → tier T0 confirmed.
- **Decisión:** `Tier.T0` con confidence 0.95+.

State machine (`soar/decision_engine/state_machine.py`):

1. Crea Incident con `incident_id=INC-2026-05-26-001` , state `RECEIVED` .
2. Persiste en Redis con key `incident:INC-2026-05-26-001` .
3. Llama al LLM Triage en paralelo (Capa 4 enrichment-only, NO bloquea decisión).
4. Transición de estado: `RECEIVED` → `PENDING_EXECUTION` .

Capa 4 LLM Triage (en paralelo, T+5 a T+8s):

- FastAPI `/triage` endpoint recibe el AlertContext con las 3 alertas.
- Llama a OpenAI GPT-4o-mini con prompt enriquecido.
- Devuelve TriageResponse: `tecnica_mitre=T1486` , `confianza=0.94` , `severidad=critical` , `runbook_aplicable="NIST 800-61 §3.4 Containment"` , `accion_recomendada="Isolate host immediately and capture memory snapshot before remediation"` .

- El Incident se actualiza en Redis con el `llm_analysis` campo.

T+8 segundos — Playbook automático

Como es T0 sin override de criticality (es una Windows VM standard, no production-critical), no requiere approval. El SOAR ejecuta el playbook directamente:

1. **Host isolation** (`soar/playbooks/host_isolation.py`): conecta vía PowerShell al Windows VM y crea regla firewall: `New-NetFirewallRule -DisplayName "ARGOS-Isolation" -Direction Outbound -Action Block` .
2. **Process kill**: identifica el PID del simulador desde el whodata FIM, ejecuta `Stop-Process -Force -Id <PID>` .
3. **Disk snapshot**: invoca Volume Shadow Copy `vssadmin create shadow /for=C:` para preservar evidencia forense.
4. Transición de estado: `PENDING_EXECUTION` → `EXECUTED` .

T+10 segundos — Notificación post-facto

El SOAR envía notificación multi-canal (per ADR-0007 v2):

- **Telegram** a los 4 chat_ids configurados: mensaje con incident_id, técnica, análisis LLM, y botón inline "Revertir" firmado con JWT.
- **Discord** webhook al server del equipo con embed coloreado por tier (T0 = rojo) y `@argos-approvers` mention.
- **Email post-facto** a `APPROVER_EMAILS` (los 4 emails del equipo) con resumen + link al audit log en OpenSearch.

T+10 a T+30 segundos — Streamlit Console muestra

La **Streamlit Approval Console** (proceso de P1 corriendo en port 8501, proyectado en pantalla) refresca cada 2 segundos vía `streamlit-autorefresh` . Muestra:

- **Panel izquierdo (Incident Card)**: badge T0 en rojo, incident_id, host afectado, técnica MITRE T1486, análisis LLM compacto.
- **Panel central (Decision Matrix)**: vacío porque es T0 auto-execute, no requirió aprobación.
- **Panel derecho (System Logic)**: estado actual `EXECUTED` , decisión `EXECUTE_ISOLATION` , política aplicada `auto-execute` , justificación "T0 confirmed: canary + L1 + L2 corroborate".
- **Panel inferior (Action Timeline)**: línea de tiempo horizontal mostrando: alert created → tier classified → playbook executed → notifications sent.

T+30 segundos — Análisis forense visible

P1 narra: "El sistema aisló el host en menos de 10 segundos. 31 archivos cifrados de 500. El canary salvó los restantes. Los 4 aprobadores tienen el incidente en su Telegram con la opción de Revertir si fuera falso. Audit log completo en OpenSearch con cada decisión justificada."

Métricas que se muestran en pantalla

- **TTD (Time to Detect):** 4.2 segundos desde T=0 hasta primera alerta válida.
- **TTI (Time to Isolate):** 8.7 segundos desde T=0 hasta playbook completado.
- **Archivos cifrados antes de containment:** 31 de 500 (94% preservados).
- **MITRE coverage:** T1083 + T1490 + T1486 cubiertos por L1, ratificados por L2 entropy spike, confirmados por canary L3.

Lo que NO se ve pero pasó

- LLM Triage tardó ~3 segundos en responder. NUNCA bloqueó la decisión de containment.
- El throttle preventivo NO se aplicó porque fue T0 (auto-execute directo). En T2 sí se habría aplicado mientras corre el countdown.
- El audit log en OpenSearch capturó: triggering layers, scores individuales, fusion logic, LLM analysis, playbook commands executed, notification channels disparados, timestamps de cada paso.

7. Glosario MITRE ATT&CK

Las técnicas relevantes al alcance del proyecto, ordenadas por categoría (tactic).

Initial Access

- **T1078 — Valid Accounts:** atacante usa credenciales legítimas. Relevante para UC-07 (escenario FP: usuario legítimo, NO atacante).
- **T1190 — Exploit Public-Facing Application:** explotación de vulnerabilidad en aplicación web expuesta. T1190.001 sub-técnica = SQL Injection. Relevante para UC-08.

Defense Evasion

- **T1562 — Impair Defenses:** atacante deshabilita herramientas de seguridad. T1562.001 sub-técnica = Disable Defender. Relevante para UC-05 (agent-kill attempt).
- **T1070 — Indicator Removal:** atacante borra rastros. T1070.001 = Clear Windows Event Logs, T1070.004 = File Deletion.

Discovery

- **T1083 — File and Directory Discovery:** ransomware enumera archivos antes de cifrar. Relevante para UC-01.

Lateral Movement

- **T1021 — Remote Services:** SMB/RDP/SSH usados por atacante para moverse. T1021.001 = RDP, T1021.002 = SMB, T1021.004 = SSH.

Command and Control

- **T1071 — Application Layer Protocol:** beacon a C2 server vía HTTP/HTTPS. T1071.001 sub-técnica = Web Protocols.

Impact (la más rica para ARGOS)

- **T1486 — Data Encrypted for Impact:** ransomware cifrando archivos. Núcleo de UC-01, UC-02, UC-03.
 - **T1490 — Inhibit System Recovery:** borrar Volume Shadow Copies, snapshots btrfs, etc. Pre-cursor a T1486.
 - **T1498 — Network Denial of Service:** ataques de red contra disponibilidad. T1498.001 = Direct Network Flood (UC-06), T1498.002 = Reflection Amplification.
 - **T1499 — Endpoint Denial of Service:** saturación de recursos del endpoint. T1499.002 = Service Exhaustion, T1499.004 = Application Exploitation.
-

8. Stack tecnológico completo

Categoría	Componente	Función	Owner principal
SIEM	Wazuh 4.7	Manager + agentes	P4
SIEM	OpenSearch	Storage de alertas y audit log	P4
SIEM	Sigma + sigma-cli	Reglas de detección	P3
Telemetría	Sysmon (Windows)	Eventos detallados de proceso/archivo/red	P4
Telemetría	auditd (Linux)	Eventos detallados de syscall	P4
Telemetría	pgAudit (PostgreSQL)	Audit log de queries	P4
Telemetría	Wazuh FIM whodata	Captura qué proceso tocó archivo (canary)	P3
Simulación	Atomic Red Team	TTPs MITRE 1-a-1	P3
Simulación	Caldera	Cadenas de ataque multi-step	P3
Simulación	Custom ransomware simulator (Python)	UC-01, UC-02, UC-03 reproducibles	P1
Simulación	hping3 + slowhttptest	UC-06 DDoS	P4
Simulación	sqlmap	UC-08 SQL injection	P4
ML	scikit-learn 1.4+	Isolation Forest + One-Class SVM	P2
ML	scipy	Shannon entropy	P2
ML	pandas + numpy	Data wrangling	P2
ML	joblib	Model serialization	P2
Backend	FastAPI	LLM Triage API + Approval API	P1
Backend	Redis 7+		

Categoría	Componente	Función	Owner principal
		Alert bus + incident state machine	P4 (deploy) / P1 (usage)
Backend	APScheduler	Timeouts + consolidation windows	P1
Backend	PyJWT	JWT signing para approval tokens	P1
Backend	Pydantic v2	argos_contracts (cross-team interfaces)	P1 (shipped)
LLM	OpenAI GPT-4o-mini	Primary backend (cloud, US-based)	P1
LLM	Llama 3.1 8B vía Ollama	Fallback (local, zero-egress)	P1
LLM	BGE-large embeddings	Retrieval en RAG	P1
Notifications	python-telegram-bot	Telegram bot con inline buttons	P1
Notifications	discord-webhook	Discord notifications con role mention	P1
Notifications	twilio	Voice DTMF escalación	P1
UI	Streamlit 1.30+	Analyst Console + Approval Console	P4 (base) / P1 (Approval)
UI	OpenSearch Dashboards	MITRE heatmap, TTD histogram, layer perf	P4
Infra	Vagrant 2.4+	Provisioning de 3 VMs	P4
Infra	VirtualBox 7.x	Hypervisor para lab local	P4
Testing	pytest + respx + fakeredis	Tests cross-module	Todos
DB	PostgreSQL 15	Activo defendido	

Categoría	Componente	Función	Owner principal
			P4 (deploy + datos sintéticos)

9. Convenciones del repo

Git workflow

- Branch principal: `main`, protegida en GitHub.
- Feature branches: `feature/<persona>/<descripcion-corta>` — ejemplo: `feature/p2/isolation-forest-baseline`.
- Commits convencionales: `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:`.
- PRs requieren: CI verde + 1 review. Pairing: P1↔P2, P3↔P4.
- Push diario obligatorio antes de las 22:00 cada día de trabajo.

Estructura del repo (post ADR-0008)



```

argos/
├── README.md           # Entry point del proyecto
├── LICENSE             # MIT
├── pyproject.toml      # Metadata + extras por módulo
├── .env.example        # Plantilla de variables (REAL .env va en .gitignore)
├── argos_contracts/   # Pydantic v2 cross-team (entregado · v1.1.0 · 69
│   ├── _mitre_data.py # MITRE_WHITELIST curado
│   ├── alert.py       # WazuhAlert, NormalizedAlert
│   ├── ml_score.py    # MLScore (P2)
│   ├── triage.py      # AlertContext, TriageResponse (P1)
│   ├── incident.py    # Incident state machine (P1)
│   ├── approval.py    # ApprovalRequest, ApprovalResponse (P1)
│   ├── enums.py       # Tier, Severity, NotificationChannelType, ...
│   └── tests/         # 69 validation tests
├── llm_triage/        # P1 – Capa 4 LLM Triage
│   ├── api/main.py    # FastAPI /triage endpoint
│   ├── llm_client/    # OpenAI primary + Llama local fallback
│   ├── rag/           # BM25 + BGE-large + RRF
│   └── prompts/       # Jinja2 templates
├── soar/              # P1 – SOAR Decision Engine + Approval API
│   ├── decision_engine/ # tier_classifier, state_machine, orchestrator
│   ├── approval/       # api, jwt_signer, consolidation
│   ├── notification/   # telegram, discord, twilio_voice, email
│   └── playbooks/      # host_isolation, process_kill, snapshot, thrott
├── ml/                # P2 – Capa 2 ML
│   ├── features/       # Feature extractors por dominio
│   ├── models/         # ransomware_ensemble, network_traffic, query_pa
│   └── consumer/       # Redis stream consumer
├── detection/         # P3 – Capa 1 Sigma rules
│   ├── sigma-rules/
│   │   ├── ransomware/ # T1486, T1490, T1083, T1562
│   │   ├── network/   # T1498, T1499 rate-based
│   │   ├── database/  # query patterns
│   │   └── webapp/     # T1190 SQLi signatures
│   └── wazuh-rules/    # Convertidas con sigma-cli
├── deception/        # P3 – Capa 3 Canary
│   ├── canary_generator.py # Genera canary files
│   └── fim-configs/      # Wazuh FIM rules

```

```

├── ui/                                # P4 (base) + P1 (Approval Console)
│   ├── streamlit_app/
│   │   ├── pages/
│   │   │   ├── 01_alert_inspection.py
│   │   │   ├── 02_approval_console.py    # P1 - PIEZA CLAVE DEL DEMO
│   │   │   └── 03_audit_forensics.py
│   └── opensearch-dashboards/            # JSON dashboards exportados
├── attack-simulation/                  # Simuladores multi-vector
│   ├── ransomware_simulator/           # P1 (LockBit-like, canary, novel)
│   ├── network_attacks/                 # P4 (hping3, slowhttptest)
│   ├── webapp_attacks/                  # P4 (sqlmap)
│   └── pg_simulator/                    # P4 (SELECT masivo para UC-07)
├── lab/                                # P4 - Vagrant + Terraform
│   ├── Vagrantfile
│   ├── provision/                       # Scripts bash/PowerShell
│   └── inventory.yaml
├── evaluation/                         # P2 + P4 - Métricas, datasets, reportes
├── docs/
│   ├── PROJECT_BRIEF.md                 # 90-second overview
│   ├── CONTEXT.md                       # Onboarding completo
│   ├── PROJECT_STATUS.md                # Honest snapshot
│   ├── EVALUATION_CRITERIA.md           # Rúbrica del curso
│   ├── data-handling.md                 # T-030 sanitization policy
│   ├── README.md                        # Mapa de docs
│   ├── architecture/                   # SAD, threat model, flow diagrams
│   ├── decisions/                       # 8 ADRs + OPEN_QUESTIONS
│   ├── use-cases/                       # 8 UCs detallados
│   └── team/                           # Manuales de integrante (este documento + 4 individuos)

```

Variables de entorno (.env)

El `.env.example` documenta TODAS las variables. Las críticas para tu rol están en tu manual individual.

Reglas globales:

- **NUNCA** commitear el `.env` — está en `.gitignore`.
- Cada integrante genera su propio `.env` partir del `.env.example`.
- Las credenciales compartidas (Telegram bot token, Discord webhook URL, OpenAI API key) las distribuye P1 vía canal privado (no en GitHub, no en Discord público).

- El `JWT_SECRET` se genera localmente con `openssl rand -hex 32`.

Convención de incident IDs

`INC-YYYY-MM-DD-NNN` donde NNN es contador diario zero-padded a 3 dígitos. Ejemplo:

`INC-2026-05-26-001`. El contador se persiste en Redis con TTL diario.

10. Quick start del entorno común

Estos pasos son comunes a TODOS los integrantes antes de empezar a implementar. Tu manual individual asume que esto está hecho.

Paso 1 — Clonar repo


BASH

```
cd ~/projects                # o donde guardes proyectos
git clone https://github.com/Enzo0rdonez/argos.git
cd argos
```

Paso 2 — Crear virtualenv Python


BASH

```
python3 -m venv .venv
source .venv/bin/activate      # Linux/macOS
# .venv\Scripts\activate      # Windows PowerShell
pip install -U pip setuptools wheel
```

Paso 3 — Instalar dependencias core + tu rol

 BASH

```
# Todos instalan estos:
pip install -e ".[dev]"

# Además según tu rol:
pip install -e ".[contracts]"      # P1 (siempre + más abajo)
pip install -e ".[ml]"            # P2
pip install -e ".[soar,llm]"      # P1 (servicios)
pip install -e ".[ui]"            # P4
```

Paso 4 — Verificar contracts

 BASH

```
pytest argos_contracts/tests/ -v
# Esperado: 69 passed
```

Si no pasan los 69 tests, revisa que el venv esté activado (`which python` debe apuntar a `.venv/bin/python`).

Paso 5 — Copiar plantilla de variables

 BASH

```
cp .env.example .env
# Editar .env con tus valores reales (tu manual individual te dice cuáles)
```

Paso 6 — Configurar git

 BASH

```
git config user.name "<Tu Nombre>"
git config user.email "<tu@email.com>"
# Crear tu branch:
git checkout -b feature/<tu-pX>/setup-inicial
```

11. Comunicación del equipo durante la implementación

Standup diario

- Hora: 9:00 AM
- Canal: Discord `#argos-standup`, llamada de voz
- Duración: 20 minutos (cada integrante ~3-4 min)
- Formato (per `docs/team/standup-template.md`):
 - Lo que cerré ayer (PRs mergeados, tests passing)
 - Lo que cierro hoy (objetivos del día)
 - Bloqueos (qué necesito de otro integrante)

Canales Discord del equipo

- `#argos-standup` — standup diario
- `#argos-help` — preguntas técnicas urgentes
- `#argos-prs` — notificación de PRs abiertos (GitHub bot)
- `#argos-incidents` — donde el bot ARGOS enviará notificaciones del demo

Reglas operativas (post ADR-0008)

1. **No tocar la doc arquitectónica.** Los docs están sincronizados con la realidad. Si descubres algo arquitectónico nuevo, abre un ADR-0009 en lugar de modificar SAD/threat model/README.

2. **No optimizar prematuramente.** El objetivo es que los 8 UCs corran end-to-end, no que sean óptimos. Performance fine-tuning queda para después del demo.
3. **Mocks son OK al inicio.** Si tu pieza depende de otra que aún no está lista, usa FakeRedis, mocked HTTP, o synthetic data. Integración real cuando ambas piezas estén listas.
4. **Verificar en lab real al menos 2 veces al día.** Cada integrante corre el flow E2E en su Vagrant antes del almuerzo y antes de pushear.
5. **Pedir ayuda en menos de 30 minutos.** Si llevas más de media hora atascado, pingueas en `#argos-help` o llamas a otro integrante. No debug solitario.
6. **Push diario obligatorio** antes de las 22:00. Si no pusheas, tu nombre aparece en el standup del día siguiente.

Claude Code / asistentes IA — política reinterpretada (ADR-0008)

Cada integrante puede usar Claude Code (o cualquier asistente IA: Cursor, GitHub Copilot, ChatGPT) como **acelerador en SU propia parte**. La condición no-negociable: cada integrante DEBE entender el código que produce con asistencia de IA y poder defenderlo en viva sin la asistencia presente. La asistencia es para velocidad, NO para reemplazar comprensión.

La regla específica que se mantiene: P1 (Enzo) no escribe código de otros integrantes con Claude Code. P1 puede asesorar y revisar, no producir el código de otros.

12. Lo que NO se entrega para el demo

Para que no te sorprendas cuando llegue el demo sin estas cosas:

- **Calibración Q5 protocol** con dataset etiquetado real (~100 ransomware + ~500 benignas).
- **UC-03 split-brain con 4 aprobadores reales** no scripted.
- **UC-05 stealth attack** end-to-end pulido.
- **PRs Sigma upstream aceptados** por SigmaHQ maintainers (depende de tiempos externos).
- **Video demo final editado.**
- **Informe técnico final.**
- **Cross-encoder reranker en RAG** (descartado del scope v1 per ADR-0001 v2).

Lo que SÍ se completa para el demo: 5 UCs corriendo end-to-end (UC-01, UC-02, UC-04, UC-06, UC-07), con UC-08 como nice-to-have, más dos rehearsals previos.

13. Documentos relacionados (referencia completa)

Si algo de este documento no quedó claro o necesitas más profundidad:

Si quieres saber...	Lee
Arquitectura completa de cada bloque	docs/architecture/SOLUTION_ARCHITECTURE_DOCUMENT.md
Por qué se tomó cada decisión arquitectónica	docs/decisions/ (ADRs 0001-0008)
Análisis de amenazas STRIDE/FMEA	docs/architecture/THREAT_MODEL.md
Detalle de los 8 UCs	docs/use-cases/USE_CASES.md
Política de manejo de datos sensibles a LLM	docs/data-handling.md
Rúbrica del curso y mapping a deliverables	docs/EVALUATION_CRITERIA.md
Estado real vs. documentado del proyecto	docs/PROJECT_STATUS.md
Flujo del sistema visualmente	docs/architecture/argos_flow.html
Tu manual individual	docs/team/manual-p<X>-<nombre>.md
Plan operacional general	docs/team/manual-equipo.md

Change log

Versión	Fecha	Cambio	Autor
1.0	2026-05-24	Initial — sección común de introducción para los 4 manuales individuales. Cubre arquitectura, UCs, ejemplo end-to-end UC-01, glosario MITRE, stack tecnológico, convenciones del repo, política Claude Code, quick start.	P1 (Enzo Ordoñez Flores)

Parte II — Manual de Diego Jara

Manual P4 — Diego Jara · Infraestructura · UI · Evaluación

Campo	Valor
Rol	Owner del lab, DB defendida, observabilidad, UI de aprobación
Owns	<code>lab/</code> (Vagrant + scripts) · <code>lab/postgres/</code> (PostgreSQL + pgAudit) · OpenSearch + Wazuh manager · <code>ui/streamlit_app/</code> · <code>ui/opensearch-dashboards/</code> · <code>evaluation/</code>
No owns	Detección/ataque (P3) · ML/LLM (P2) · SOAR Engine (P1)
Outputs blocking	Lab funcional (TODOS dependen) · Redis disponible · Streamlit Approval Console (centerpiece UC-04)
Entrega final	13 de junio de 2026 — eres el operador del demo

0. Tu charter

Si tu laptop arranca el lab en menos de 5 minutos con `vagrant up`, el equipo trabaja. Si no, todos quedan bloqueados. Eres también el operador del demo: clickeas los botones en vivo mientras P1 narra. Tu UI es la pantalla que el profesor mirará 12 minutos seguidos.

0.1 Cómo leer cada sub-sección

Cada componente sigue: **Contexto** → **Pasos manuales** si aplica → **Comandos** → **Salida esperada** → **Verificación** → Si algo falla.

Fase 1 — Cimientos: el lab

1.1 Prerequisites en tu laptop

Comandos


BASH

```
VBoxManage --version  
vagrant --version  
df -h ~ | tail -1  
free -h | head -2
```

Salida esperada

SALIDA ESPERADA

```
7.0.x  
Vagrant 2.4.x  
/dev/sda1      500G   100G   400G   20% /home/usuario  
              total  used  free  shared  buff/cache  available  
Mem:          15Gi   6.0Gi  3.0Gi  400Mi   6.0Gi      8.0Gi
```

Verificación

VERIFICACIÓN

```
VBoxManage --version | awk -F'r' '{print $1}' | awk -F'.' '{exit !($1 ≥ 7)}' && echo  
test $(df -BG ~ | tail -1 | awk '{print $4}' | tr -d 'G') -ge 60 && echo "Disk OK"
```

Esperado:

SALIDA ESPERADA

```
VBox 7+ OK  
Disk OK
```

Si algo falla

Síntoma	Causa	Fix
<code>VBoxManage:</code> <code>command not found</code>	VirtualBox no instalado o PATH	Instala desde https://www.virtualbox.org/wiki/Downloads
Disco < 60 GB libre	VMs no caben	Limpia downloads o usa SSD externo (USB-C 256 GB ≈ USD 30)
RAM < 16 GB	Sólo puedes levantar una VM a la vez	Levanta <code>lab-manager</code> + <code>linux-victim</code> , deja Windows abajo y comparte ese rato con P3

1.2 Clonar repo

Comandos

 BASH

```
mkdir -p ~/code && cd ~/code
git clone git@github.com:Enzo0rdonez/argos.git
cd argos
python3 -m venv .venv
source .venv/bin/activate
pip install -e ./argos_contracts
pip install -r ui/requirements.txt
pip install -r lab/requirements.txt
```

Verificación

 VERIFICACIÓN

```
python -c "import argos_contracts, streamlit, redis; print('imports OK')"
```

Esperado:

SALIDA ESPERADA

imports OK

1.3 Vagrantfile

Contexto

Tres VMs: Linux manager (Wazuh + OpenSearch + Redis), Linux víctima (Postgres + Wazuh agent + auditd), Windows víctima (Sysmon + Wazuh agent).

lab/Vagrantfile




```
# Lab ARGOS: 1 Windows victim, 1 Linux victim + Postgres, 1 Linux manager.  
# Red interna 192.168.56.0/24
```

```
Vagrant.configure("2") do |config|
```

```
  config.vm.define "lab-manager" do |m|  
    m.vm.box      = "bento/ubuntu-22.04"  
    m.vm.hostname = "lab-manager"  
    m.vm.network "private_network", ip: "192.168.56.10"  
    m.vm.synced_folder "..", "/vagrant"  
    m.vm.provider "virtualbox" do |vb|  
      vb.memory = 4096  
      vb.cpus   = 2  
    end  
    m.vm.provision "shell", path: "provision/manager.sh"  
  end
```

```
  config.vm.define "linux-victim" do |l|  
    l.vm.box      = "bento/ubuntu-22.04"  
    l.vm.hostname = "linux-victim"  
    l.vm.network "private_network", ip: "192.168.56.21"  
    l.vm.synced_folder "..", "/vagrant"  
    l.vm.provider "virtualbox" do |vb|  
      vb.memory = 3072  
      vb.cpus   = 2  
    end  
    l.vm.provision "shell", path: "provision/linux_victim.sh"  
  end
```

```
  config.vm.define "windows-victim" do |w|  
    w.vm.box      = "gusztavvargadr/windows-10"  
    w.vm.hostname = "win-victim"  
    w.vm.network "private_network", ip: "192.168.56.20"  
    w.vm.provider "virtualbox" do |vb|  
      vb.memory = 4096  
      vb.cpus   = 2  
    end  
    w.vm.provision "shell", path: "provision/windows_victim.ps1"  
  end  
end
```

Estructura

SALIDA ESPERADA

```
lab/
├── Vagrantfile          ← arriba
├── provision/
│   ├── manager.sh      ← Wazuh + OpenSearch + Redis + Docker
│   ├── linux_victim.sh ← Wazuh agent + Postgres + auditd
│   └── windows_victim.ps1 ← Sysmon + Wazuh agent (vía P3)
└── postgres/
    └── init.sql         ← schema audit
```

1.4 `vagrant up` y verificación

Pasos manuales

1. La primera vez: `vagrant up --provider=virtualbox` tarda 10-15 min (descarga boxes + provision).
2. Reinicios sucesivos (después de `vagrant halt`): ~3-5 min.
3. Si algo se rompe sin causa clara: `vagrant destroy <vm> && vagrant up <vm>`.

Comandos


BASH

```
cd lab/
vagrant up --provider=virtualbox

vagrant status
```

Salida esperada

SALIDA ESPERADA

```
⇒ lab-manager: Importing base box 'bento/ubuntu-22.04' ...  
... (mucho output)  
⇒ windows-victim: Machine booted and ready!  
⇒ linux-victim: Machine booted and ready!  
⇒ lab-manager: Machine booted and ready!
```

Current machine states:

lab-manager	running (virtualbox)
linux-victim	running (virtualbox)
windows-victim	running (virtualbox)

Verificación — smoke tests

VERIFICACIÓN

```
curl -sk -u wazuh:wazuh https://192.168.56.10:55000/?pretty | jq .data.title  
curl -sk -u admin:admin https://192.168.56.10:9200/_cluster/health | jq .status  
redis-cli -h 192.168.56.10 ping  
psql postgresql://argos:argos@192.168.56.21:5432/argos_audit -c "SELECT 1;"
```

Esperado:

SALIDA ESPERADA

```
"Wazuh API REST"  
"green"  
PONG  
?column?  
-----  
1  
(1 row)
```

Si algo falla

Síntoma	Causa	Fix
<code>Vagrant could not detect VBoxManage</code>	VirtualBox no en PATH	Agrega <code>C:\Program Files\Oracle\VirtualBox</code> (Windows) o <code>/usr/local/bin</code> (macOS) al PATH
VM Windows timeout WinRM	Provision Windows tarda	<code>vagrant reload windows-victim --provision</code> ; si persiste: destroy + up
OpenSearch cluster red	OOM o disk full	<code>docker logs opensearch</code> y verifica <code>OPENSEARCH_JAVA_OPTS=-Xms1g -Xmx1g</code>

Checklist Fase 1

#	Check	OK
1	VirtualBox 7+ · Vagrant 2.4+ · 60 GB disk	☐
2	3 VMs <code>running</code>	☐
3	Wazuh API responde con <code>"Wazuh API REST"</code>	☐
4	OpenSearch health green/yellow	☐
5	Redis pingable desde host	☐
6	Postgres conectable desde host	☐
7	<code>vagrant up</code> desde halted ≤ 5 min	☐

Fase 2 — Postgres defendido + observabilidad

2.1 PostgreSQL 15 + pgAudit

Contexto

Postgres es el activo defendido. pgAudit registra todos los SELECT/INSERT/DDL para el audit log y como input a las Sigma rules de DB (UC-07, UC-08).

lab/postgres/init.sql



```

-- DB de aplicación (target del ataque)
CREATE DATABASE app_prod;

-- DB de auditoría (consumida por audit sink de P1)
CREATE DATABASE argos_audit;

\c argos_audit

CREATE TABLE IF NOT EXISTS audit_incidents (
    incident_id    text PRIMARY KEY,
    tier            text NOT NULL,
    severity       text NOT NULL,
    host           text NOT NULL,
    technique      text NOT NULL,
    created_at     timestampz NOT NULL,
    final_outcome  text,
    final_policy   text,
    final_at       timestampz,
    payload        jsonb NOT NULL
);

CREATE TABLE IF NOT EXISTS audit_responses (
    id              bigserial PRIMARY KEY,
    incident_id     text REFERENCES audit_incidents(incident_id) ON DELETE CASCADE,
    approver_id     text NOT NULL,
    channel         text NOT NULL,
    decision        text NOT NULL,
    received_at     timestampz NOT NULL
);

CREATE INDEX idx_audit_incidents_created ON audit_incidents(created_at DESC);
CREATE INDEX idx_audit_responses_incident ON audit_responses(incident_id);

CREATE USER argos WITH PASSWORD 'argos';
GRANT ALL PRIVILEGES ON DATABASE argos_audit TO argos;
GRANT ALL ON ALL TABLES IN SCHEMA public TO argos;

CREATE USER analyst WITH PASSWORD 'analyst_pwd';
GRANT CONNECT ON DATABASE app_prod TO analyst;

```

Pasos manuales — habilitar pgAudit

1. En linux-victim, instalar `postgresql-15-pgaudit`.
2. Agregar `pgaudit` a `shared_preload_libraries` en `postgresql.conf`.
3. Configurar `pgaudit.log` y `listen_addresses`.
4. Permitir conexiones desde 192.168.56.0/24 en `pg_hba.conf`.
5. Reiniciar Postgres.
6. Crear la extensión en cada DB.

Comandos (en linux-victim, via `vagrant ssh`)

BASH

```
sudo apt install -y postgresql-15-pgaudit

sudo tee -a /etc/postgresql/15/main/postgresql.conf << 'EOF'
shared_preload_libraries = 'pgaudit'
pgaudit.log = 'read, write, ddl, role'
pgaudit.log_relation = on
pgaudit.log_parameter = on
pgaudit.log_catalog = off
listen_addresses = '*'
EOF

sudo tee -a /etc/postgresql/15/main/pg_hba.conf << 'EOF'
host    all    all    192.168.56.0/24    md5
EOF

sudo systemctl restart postgresql

sudo -u postgres psql -c "CREATE EXTENSION IF NOT EXISTS pgaudit;" -d app_prod
sudo -u postgres psql -c "CREATE EXTENSION IF NOT EXISTS pgaudit;" -d argos_audit
```

Verificación

VERIFICACIÓN

```

sudo -u postgres psql -d app_prod -c "SELECT extname, extversion FROM pg_extension"
sudo -u postgres psql -d app_prod -c "SELECT 1;"
sudo tail -3 /var/log/postgresql/postgresql-15-main.log | grep -i pgaudit

```

Esperado:

SALIDA ESPERADA

```

 extname | extversion
-----+-----
 pgaudit | 1.7
(1 row)

```

```
LOG: AUDIT: SESSION,READ,SELECT,SELECT 1;
```

Si algo falla

Síntoma	Causa	Fix
<code>ERROR: pgaudit must be in shared_preload_libraries</code>	Modificaste <code>postgresql.conf</code> pero no reiniciaste	<code>sudo systemctl restart postgresql</code>
<code>psql: FATAL: password authentication failed</code> desde host	<code>pg_hba.conf</code> no permite tu rango	Confirma <code>host all all 192.168.56.0/24 md5</code> y <code>systemctl reload postgresql</code>
<code>Permission denied: postgresql.log</code>	Permisos	<code>sudo chmod 644 /var/log/postgresql/postgresql-15-main.log</code>

2.2 OpenSearch + OpenSearch Dashboards (Docker Compose)

Contexto

OpenSearch indexa los alerts del Wazuh manager y aporta los 3 dashboards visuales (Alerts Timeline, MITRE Heatmap, Layer Performance).

Pasos manuales (en `provision/manager.sh`)

1. Instalar Docker + compose plugin.
2. Crear `/opt/opensearch/docker-compose.yml` .
3. `docker compose up -d` .
4. Esperar 30 s a que cluster esté ready.

Comandos (en lab-manager, vía `vagrant ssh`)



BASH

```

sudo apt-get install -y docker.io docker-compose-plugin
sudo mkdir -p /opt/opensearch && cd /opt/opensearch

sudo tee docker-compose.yml << 'EOF'
services:
  opensearch:
    image: opensearchproject/opensearch:2.11.0
    container_name: opensearch
    environment:
      - cluster.name=argos-cluster
      - node.name=opensearch-node1
      - discovery.type=single-node
      - bootstrap.memory_lock=true
      - OPENSEARCH_JAVA_OPTS=-Xms1g -Xmx1g
    ulimits:
      memlock: { soft: -1, hard: -1 }
    volumes:
      - opensearch-data:/usr/share/opensearch/data
    ports: ["9200:9200", "9600:9600"]
  dashboards:
    image: opensearchproject/opensearch-dashboards:2.11.0
    container_name: opensearch-dashboards
    environment:
      - 'OPENSEARCH_HOSTS=["https://opensearch:9200"]'
    ports: ["5601:5601"]
volumes:
  opensearch-data:
EOF

sudo docker compose up -d
sleep 30

```

Verificación

VERIFICACIÓN

```

curl -sk https://localhost:9200/_cluster/health | jq .status
curl -s http://localhost:5601/api/status | jq .status.overall.state 2>/dev/null || c

```

Esperado:

SALIDA ESPERADA

```
"green"  
"green"
```

Si algo falla

Síntoma	Causa	Fix
<code>bootstrap.memory_lock=true</code> warning	<code>ulimits memlock</code> no aplicado	Reinicia el container: <code>sudo docker compose restart</code>
<code>cluster.status: red</code>	OOM	Bajar heap: <code>OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m</code>
Dashboards no carga (404)	Aún arrancando	Espera 30-60 s más y reintenta

2.3 Redis

Comandos (en lab-manager)


BASH

```
sudo apt-get install -y redis-server  
sudo sed -i 's/^bind 127.0.0.1.*/bind 0.0.0.0/' /etc/redis/redis.conf  
sudo sed -i 's/^protected-mode yes/protected-mode no/' /etc/redis/redis.conf  
sudo systemctl restart redis-server
```

Verificación



VERIFICACIÓN

```
redis-cli -h 192.168.56.10 ping
redis-cli -h 192.168.56.10 config get save
```

Esperado:

SALIDA ESPERADA

```
PONG
1) "save"
2) "3600 1 300 100 60 10000"
```

Checklist Fase 2

#	Check	OK
1	Postgres con pgAudit funcional	☐
2	SELECT genera entrada AUDIT: en log	☐
3	OpenSearch health green	☐
4	Dashboards UI abre en http://192.168.56.10:5601	☐
5	Redis pingable desde host	☐

Fase 3 — Bridge Wazuh→Redis + Streamlit UI

3.1 Bridge `alerts.json` → `events:raw_wazuh`

Contexto

P3 produce alertas en `/var/ossec/logs/alerts/alerts.json`. P2 las consume desde Redis Stream. Tu bridge cierra ese gap: tail-ea el archivo y empuja a Redis.

`lab/bridge/wazuh_to_redis.py`



```

"""Tail alerts.json del Wazuh manager y push a Redis stream events:raw_wazuh."""

from __future__ import annotations
import json, time, os
from pathlib import Path
import redis

ALERTS_FILE = Path("/var/ossec/logs/alerts/alerts.json")
STREAM      = "events:raw_wazuh"

def tail_f(path: Path):
    with path.open() as f:
        f.seek(0, 2) # ir al final
        while True:
            line = f.readline()
            if not line:
                time.sleep(0.5); continue
            yield line

def main():
    r = redis.from_url(os.environ.get("REDIS_URL", "redis://localhost:6379/0"))
    for line in tail_f(ALERTS_FILE):
        try:
            alert = json.loads(line)
            if alert.get("rule", {}).get("level", 0) < 5:
                continue
            payload = {
                "host": alert.get("agent", {}).get("name", "unknown"),
                "mitre_technique": alert.get("rule", {}).get("mitre", {}).get("technique", "unknown"),
                "syscalls_per_min": alert.get("data", {}).get("syscalls_per_min", 0),
                "files_touched_per_min": alert.get("data", {}).get("files_touched_per_min", 0),
                "entropy_of_written_bytes": alert.get("data", {}).get("entropy", 0.0),
                "network_kbps": alert.get("data", {}).get("network_kbps", 0),
                "command_line": alert.get("data", {}).get("win", {}).get("event_command_line", "unknown"),
                "raw": alert,
            }
            r.xadd(STREAM, {"data": json.dumps(payload)})
        except Exception as e:
            print(f"[!] {e}", flush=True)

```

```
if __name__ == "__main__":
    main()
```

Pasos manuales — systemd service

1. Copiar el unit file a `/etc/systemd/system/argos-bridge.service`.
2. `systemctl daemon-reload`.
3. `systemctl enable --now argos-bridge`.
4. Verificar `systemctl status`.

Comandos (en lab-manager)


BASH

```
sudo tee /etc/systemd/system/argos-bridge.service << 'EOF'
[Unit]
Description=ARGOS Wazuh→Redis bridge
After=wazuh-manager.service

[Service]
ExecStart=/usr/bin/python3 /vagrant/lab/bridge/wazuh_to_redis.py
Restart=always
Environment=REDIS_URL=redis://localhost:6379/0

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now argos-bridge
systemctl status argos-bridge --no-pager
```

Salida esperada

SALIDA ESPERADA

- `argos-bridge.service` - ARGOS Wazuh→Redis bridge
Loaded: loaded (/etc/systemd/system/argos-bridge.service; enabled)
Active: active (running) since ...

Verificación (coordinar con P3 para disparar canary)

 VERIFICACIÓN

```
redis-cli -h 192.168.56.10 XLEN events:raw_wazuh
```

Esperado:

SALIDA ESPERADA

1

(Crece tras cada alerta de level $\geq$ 5.)

3.2 Streamlit Approval Console (CENTERPIECE)

Contexto

Tab 2 de la UI es el centerpiece visual del demo UC-04. Muestra Incident card, Decision Matrix con estado por aprobador, Consolidation Window countdown, y banner final.

`ui/streamlit_app/app.py`

 PYTHON


```

"""Streamlit Analyst UI – 3 tabs (Alert Inspection / Approval Console / Audit)."""

import os, time, json

import streamlit as st
from streamlit_autorefresh import st_autorefresh
import redis

st.set_page_config(page_title="ARGOS Analyst", layout="wide",
                    initial_sidebar_state="collapsed")
st_autorefresh(interval=1500, key="poll")

r = redis.from_url(os.environ.get("REDIS_URL", "redis://localhost:6379/0"),
                  decode_responses=True)

def load_active_incidents() → list[dict]:
    keys = sorted(r.scan_iter(match="incident:inc-*"), reverse=True)[:20]
    return [json.loads(r.get(k)) for k in keys if r.get(k)]

def tier_color(tier: str) → str:
    return {"T0": "#E53935", "T1": "#FB8C00",
            "T2": "#FDD835", "T3": "#1E88E5"}.get(tier, "#888")

tab1, tab2, tab3 = st.tabs(["Alert Inspection",
                             "Approval Console",
                             "Audit & Forensics"])

with tab2:
    st.title("Approval Workflow Console")
    incidents = [i for i in load_active_incidents()
                  if i.get("tier") == "T2" and i.get("final_decision") is None]

    if not incidents:
        st.info("No active T2 incidents waiting for approval.")
    else:
        inc = incidents[0]
        st.markdown(
            f"<div style='background:{tier_color(inc['tier'])};padding:1rem;"
            f"border-radius:8px;color:white;'>"
            f"<h2>Incident {inc['incident_id']} – Tier {inc['tier']}</h2>"

```

```

        f"<p>Host: <b>{inc['host']['hostname']}

```

Comandos


BASH

```

cd ui/
pip install -r requirements.txt
streamlit run streamlit_app/app.py --server.address 0.0.0.0 --server.port 8501

```

Salida esperada

SALIDA ESPERADA

You can now view your Streamlit app in your browser.

Local URL: `http://localhost:8501`

Network URL: `http://192.168.x.x:8501`

Verificación

VERIFICACIÓN

```
curl -s http://localhost:8501/_stcore/health
```

Esperado:

SALIDA ESPERADA

ok

Abre `http://localhost:8501`, ve a tab Approval Console. Si Redis vacío, muestra `"No active T2 incidents waiting for approval."`

3.3 OpenSearch Dashboards exportados a NDJSON

Pasos manuales — crear los 3 dashboards en la UI

1. Abre `http://192.168.56.10:5601` y autentícate (admin/admin).
2. Crear index pattern `wazuh-alerts-*`.
3. Crear **Alerts Timeline** (visualización tipo histograma por `@timestamp` + `rule.level`).
4. Crear **MITRE Heatmap** (heatmap `rule.mitre.technique` vs día).

5. Crear **Layer Performance** (línea con conteo por `layer_origin` por minuto).
6. Exportar los 3 a NDJSON.

Comandos (export)


BASH

```
for d in "Alerts Timeline" "MITRE Heatmap" "Layer Performance"; do
  slug=$(echo "$d" | tr '[:upper:]' '[:lower:]' | tr ' ' '-')
  echo "Exporting $d..."
  curl -X POST "http://localhost:5601/api/saved_objects/_export" \
    -H "osd-xsrf: true" \
    -H "Content-Type: application/json" \
    -d '{"type": "dashboard", "includeReferencesDeep": true}' \
    > "ui/opensearch-dashboards/${slug}.ndjson"
done
```

Verificación


VERIFICACIÓN

```
ls ui/opensearch-dashboards/*.ndjson | wc -l
head -1 ui/opensearch-dashboards/alerts-timeline.ndjson | jq .type
```

Esperado:

SALIDA ESPERADA

```
3
"dashboard"
```

Checklist Fase 3

#	Check	OK
1	<code>argos-bridge</code> service active	☐
2	Streamlit Approval Console renderiza con incident demo	☐
3	3 OpenSearch dashboards exportados a NDJSON	☐

Fase 4 — Rehearsal + operador del demo

4.1 Rehearsal del rol de operador

Contexto

El día del demo eres el operador. P1 narra, P2 y P3 aprueban en sus celulares, tú ejecutas los comandos. Ensaya esto al menos 2 veces solo y 1 vez con el equipo.

Comandos pre-preparados (uno por terminal, listas para Enter)



```
# Terminal 1 (UC-01)
vagrant ssh linux-victim -c 'python /vagrant/attack-simulation/ransomware_simulator.py'

# Terminal 2 (UC-02)
vagrant ssh linux-victim -c 'python /vagrant/attack-simulation/ransomware_simulator.py'

# Terminal 3 (UC-04)
vagrant ssh linux-victim -c 'python /vagrant/attack-simulation/ransomware_simulator.py'

# Terminal 4 (UC-06 opcional si hay tiempo)
sudo timeout 5 hping3 -S --flood -p 80 192.168.56.21
```

Tips del operador

- Tener 4 terminales con los comandos **pre-tipeados** (no tipear en vivo).
- Pantalla del proyector = sólo Streamlit Tab 2. NO mostrar terminales.
- Si algo falla, **no entres en pánico**: `make demo-reset` y reanudar en el último UC.

4.2 Makefile ayudante

Makefile (raíz del repo)



```
.PHONY: demo-up demo-down demo-reset soar-restart bridge-restart llm-restart

demo-up:
    cd lab && vagrant up
    @echo "Lab up. Waiting 10s for services..."
    @sleep 10

demo-down:
    cd lab && vagrant halt

demo-reset:
    redis-cli -h 192.168.56.10 --scan --pattern 'incident:*' | xargs -r redis-cli -l
    vagrant ssh lab-manager -c "sudo systemctl restart argos-bridge"
    @echo "Demo state reset."

soar-restart:
    pkill -f 'soar.decision_engine.consumer' || true
    pkill -f 'uvicorn soar.approval_api' || true
    sleep 1
    nohup python -m soar.decision_engine.consumer > /tmp/soar-consumer.log 2>&1 &
    nohup uvicorn soar.approval_api.main:app --port 8001 > /tmp/soar-api.log 2>&1 &

bridge-restart:
    vagrant ssh lab-manager -c "sudo systemctl restart argos-bridge"

llm-restart:
    pkill -f 'ml.consumer' || true
    sleep 1
    nohup python -m ml.consumer > /tmp/ml.log 2>&1 &
```

Verificación

 VERIFICACIÓN

```
make demo-up
sleep 5
make demo-reset
```

Esperado:

SALIDA ESPERADA

```
... vagrant logs ...  
Lab up. Waiting 10s for services...  
Demo state reset.
```

4.3 Hot spare en laptop de P1

Pasos manuales

P1 tiene un clon del lab. Tu trabajo es asegurar que él pueda hacer `cd ~/code/argos/lab && vagrant up` y obtener un lab idéntico. Coordina una prueba conjunta:

1. Tú apagas tu lab (`vagrant halt`).
2. P1 prende el suyo.
3. Cambia env var `WAZUH_HOST=p1-espejo.local` en los servicios SOAR/ML.
4. Corren un mini UC-01.
5. Documenta el tiempo de switchover en `docs/LESSONS_LEARNED.md` . Objetivo: ≤ 5 min.

4.4 Video respaldo

Comandos


BASH

```
# Grabar pantalla completa de tu rehearsal final (OBS o vokoscreenNG)  
# Output: docs/team/argos-demo-rehearsal.mp4  
# NO commitear si > 100 MB; sube a Drive y comparte link en standup.  
obs-studio  # o vokoscreenNG
```


Verificación

VERIFICACIÓN

```
ls -lah docs/team/argos-demo-rehearsal.mp4 2>/dev/null || echo "video pendiente"
```

4.5 Checklist pre-demo (T-2 h)

#	Check	OK
1	<code>vagrant status</code> → 3 running	☐
2	Wazuh API health 200	☐
3	OpenSearch health green	☐
4	Streamlit Approval Console abre y muestra "No active"	☐
5	Bridge service activo	☐
6	Postgres conectable	☐
7	Hot spare P1 también <code>running</code>	☐
8	Video respaldo accesible offline	☐
9	Cargador laptop + cable HDMI/USB-C al proyector	☐
10	4 terminales preparadas con comandos UC-01..04	☐

Apéndice A — Troubleshooting

#	Síntoma	Diagnóstico	Fix
A. 1	<code>vagrant up</code> falla con <code>Vagrant could not detect VBoxManage</code>	PATH	Agrega ruta de VirtualBox al PATH
A. 2	VM Windows no arranca (WinRM timeout)	Provision pesado	<code>vagrant reload windows-victim --provision</code> ; si persiste: <code>destroy + up</code>
A. 3	OpenSearch OOM	Heap insuficiente	Subir <code>OPENSEARCH_JAVA_OPTS=-Xms1500m -Xmx1500m</code>
A. 4	Bridge no recibe alerts	<code>alerts.json</code> no existe (Wazuh down)	<code>sudo /var/ossec/bin/wazuh-control status</code> y restart si parado
A. 5	Streamlit no refresca	Polling pausado	F5 manual; los datos están en Redis
A. 6	Postgres <code>FATAL: password authentication failed</code>	<code>pg_hba.conf</code> no permite tu rango	Confirma <code>host all all 192.168.56.0/24 md5</code>
A. 7	Wazuh agent no aparece en manager	Server IP mal config	Editar <code><server-ip></code> en agent <code>ossec.conf</code> y restart

Apéndice B — Comandos de emergencia



```
# Lab caído mid-demo
make demo-reset

# Reload completo del manager (último recurso, ~30s downtime)
vagrant reload lab-manager

# Switch a lab espejo de P1
export WAZUH_HOST=p1-espejo.local
export REDIS_URL=redis://p1-espejo.local:6379/0
make soar-restart

# Postgres lleno
psql -U argos -d argos_audit -c "DELETE FROM audit_incidents WHERE created_at < NOW"

# OpenSearch indexes lentos
curl -X POST 'http://localhost:9200/_cache/clear?pretty'

# Streamlit muerto
pkill -f streamlit
nohup streamlit run ui/streamlit_app/app.py --server.port 8501 > /tmp/streamlit.log
```

Apéndice C — Referencias cruzadas

Cuando estés en...	Lee
<code>lab/Vagrantfile</code>	SAD §13, <code>docs/team/manual-equipo.md</code> §1
<code>lab/postgres/</code>	Manual P1 §3.2
<code>ui/streamlit_app/</code>	SAD §9.2, ADR-0006
<code>evaluation/</code>	EVALUATION_CRITERIA §1.3

Change log

Versión	Fecha	Cambio
3.0	2026-05-24	Reestructurado: Contexto → Pasos manuales → Comandos → Salida → Verificación → Si algo falla. Vagrantfile, pgAudit, bridge, Streamlit completos. Bloques bash/sql/yaml listos para copy buttons. Sin referencias temporales. Renombrado de <code>sprint-week-1-p4-diego.md</code> .